

Pas Cpp

Pascal 转 C++ 急救

用来急救，不多废话。

▼ 药方食用提示

本急救贴可以让您充分了解以下内容（对应 [C++ 语法快速提要](#)）：

- 基本语法（块语句、注释、导入库、简单输入输出、声明变量、赋值.....）
- C++ 的 Hello World 与 A+B Problem 写法与解释

[对应语法](#) 部分较为紧凑，正式食用可能需要额外参考资料（已给出）。此部分不包括指针与 C 风格数组的介绍，也没有结构体、运算符重载等等。

[重要不同之处](#) 部分为 C++ 的语法特点，也是 Pascal 转 C++ 时会碰到的坑。

如要快速查找，请见附录：

- [附 A: Pascal 与 C++ 运算符与数学函数语法对比表](#)
- [附 B: 文章检索](#) - 按 C++ 语句语法索引

C++ 快速安装与环境配置

注意：这里假设使用的系统是 Windows。

方式一：使用 IDE

以下 IDE 选择一个即可：

- [Dev-C++](#)
- [Code::Blocks](#)

方式二：使用 代码编辑器 + 编译器 + 调试器

- [VS Code](#)

Visual Studio Code 官方网站上有文档解释如何进行 C++ 的配置。一般而言 VS Code 搭配插件使用更方便，见 [VS Code 的官方网站](#)。

C++ 语法快速提要 Start Here

C++ 程序都是从 `main` 这个部分开始运行的。

大括号表示块语句的开始与结束：`{` 就相当于 Pascal 里面的 `begin`，而 `}` 就相当于 `end`。

注意，和 Pascal 一样，C++ 每句话结束要加分号 `;`，不过大括号结尾不需要有分号，而且程序结束末尾不用打句号 `.`。

`//` 表示行内注释，`/* */` 表示块注释。

按照惯例，看看 Hello World 吧。

Hello World：第一个 C++ 程序

```
1 #include <iostream> // 导入 iostream 库
2
3 int main() // main 部分
4 {
5     std::cout << "Hello World!" << std::endl;
6
7     return 0;
8 }
```

然后编译运行一下，看看结果。

简要解释

第一行，`#include <iostream>` 的意思是，导入 `iostream` 这个库。

► Pascal 的库文件

看到第三行的 `main` 吗？程序从 `main` 开始执行。

接下来最重要的一句话是

```
1 std::cout << "Hello World!" << std::endl;
```

`std::cout` 是输出（`cout` 即 C-out）的命令。你可能看过有些 C++ 程序中直接写的是 `cout`。

► 有关 `std::` 前缀

中间的 `<<` 很形象地表示流动，其实它就是表示输出怎么「流动」的。这句代码的意思就是，`"Hello World!"` 会先被推到输出流，之后 `std::endl` 再被推到输出流。

而 `std::endl` 是输出换行（`endl` 即 end-line）命令，这与 Pascal 的 `writeln` 类似，不过 C++ 里面可没有 `coutln`。Pascal 与 C++ 的区别在于，`write('Hello World!')` 等价于 `std::cout << "Hello World!"`，而 `writeln('Hello World!')` 等价于 `std::cout << "Hello World!" << std::endl`。

此处 `"Hello World!"` 是字符串，Pascal 中字符串都是用单引号，不能用双引号，而 C++ 的字符串必须用双引号。C++ 中单引号包围的字符会有别的含义，后面会再提及的。

好了，到这里 Hello World 应该解释的差不多了。

可能有同学会问，后面那个 `return 0` 是什么意思？那个 `int main()` 是啥意思？先别管它，一开始写程序的时候先把它当作模板来写吧（这里也是用模板写的）。因为入门时并不会用到 `main` 中参数，所以不需要写成 `int main(int argc, char const *argv[])`。

简单练习

1. 试着换个字符串输出。
2. 试着了解转义字符。

A+B Problem：第二个 C++ 程序

经典的 A+B Problem。

```

1 #include <iostream>
2
3 int main() {
4     int a, b, c;
5
6     std::cin >> a >> b;
7
8     c = a + b;
9
10    std::cout << c << std::endl;
11
12    return 0;
13 }

```

注：代码空行较多，若不习惯可去掉空行。

简要解释

`std::cin` 是读入（`cin` 即 C-in），`>>` 也与输出语法的类似。

这里多出来的语句中最重要的是两个，一个是变量声明语句

```

1 int a, b, c;

```

你可能习惯于 Pascal 里面的声明变量

```

1 var
2 a, b, c: integer;

```

C++ 的声明是直接以数据类型名开头的，在这里，`int`（整型）开头表示接下来要声明变量。

接着一个最重要的语句就是赋值语句

```

1 c = a + b;

```

这是 Pascal 与 C++ 语法较大的不同，**这是个坑**：Pascal 是 `:=`，C++ 是 `=`；而 C++ 判断相等是 `==`。

C++ 也可直接在声明时进行变量初始化赋值

```

1 int a = 0, b = 0, c = 0;

```

简单练习

1. 重写一遍代码，提交到 OJ 上，并且 AC。
2. 更多的输入输出语法参考 [这节内容](#)，并试着了解 C++ 的格式化输出。

结束语与下一步

好了，到现在为止，你已经掌握了一些最基本的东西了，剩下就是找 Pascal 和 C++ 里面对应的语法和不同的特征。

不过在此之前，强烈建议先看 [变量作用域：全局变量与局部变量](#)，也可使用 [附 B：文章检索](#) 查阅阅读。

请善用 `Alt+←` 与 `Alt+→` 返回跳转。

对应语法 Syntax

变量 Variable

基本数据类型 Fundamental types

C++ 与 Pascal 基本上差不多，常见的有

- `bool` Boolean 类型
- `int` 整型
- 浮点型
 - `float`
 - `double`
- `char` 字符型
- `void` 无类型

C++ 的单引号是专门用于表示单个字符的（字符型），比如 `'a'`，而字符串（字符型数组）必须要用双引号。

C++ 还要很多额外的数据类型，请参考更多资料。

扩展阅读：

- [基础类型 - cppreference.com](http://cppreference.com)

常量声明 Constant

```
1 const double PI = 3.1415926;
```

若不清楚有关宏展开的问题，建议使用常量，而不用宏定义。

运算符 Operator

请直接参考

- [附 A: Pascal 与 C++ 运算符与数学函数语法对比表](#)
- [运算 - OI Wiki](#)

条件

`if` 语句

```
1 if (a = b) and (a > 0) and (b > 0) then
2     begin
3         b := a;
4     end
5 else
6     begin
7         a := b;
8     end;
```

```
1 if (a == b && a > 0 && b > 0) {
2     b = a;
3 } else {
4     a = b;
5 }
```

布尔运算与比较

- `and` -> `&&`
- `or` -> `||`
- `not` -> `!`
- `=` -> `==`
- `<>` -> `!=`

注释:

1. Pascal 中 `and` 与 C++ 中 `&&` 优先级不同, C++ 不需要给判断条件加括号。
2. Pascal 中判断相等是 `=`, 赋值是 `:=`; C++ 中判断相等是 `==`, 赋值是 `=`。
3. 如果在 `if` 语句的括号内写了 `a = b` 而不是 `a == b`, 程序不会报错, 而是会把 `b` 赋值给 `a`, 并使赋值表达式 `a = b` 整体具有 `a` 在赋值操作完成后的值。
4. C++ 不需要思考到底要不要在 `end` 后面加分号。
5. C++ 布尔运算中, 非布尔值可以自动转化为布尔值。

▼ 易错提醒

特别注意: 不要把 `==` 写成 `= !`

由于 C/C++ 比 Pascal 语法灵活, 如果在判断语句中写了 `if (a=b) {`, 那么程序会顺利运行下去, 因为 C++ 中 `a=b` 是有返回值的。

case 与 switch

用到得不多, 此处不详细展开。

需要注意: C++ 没有 `1..n`, 也没有连续不等式 (比如 `1 < x < 2`)。

循环 Loop

以下三种循环、六份代码实现的功能是一样的。

while 循环

`while` 很相似。(C++ 此处并非完整程序, 省略一些框架模板, 后同)

```
1 var i: integer;
2
3 begin
4     i := 1;
5     while i <= 10 do
6         begin
7             write(i, ' ');
8             inc(i); // 或者 i := i + 1;
9         end;
10 end.
```

```
1 int i = 1;
2 while (i <= 10) {
3     std::cout << i << " ";
4     i++;
5 }
```

for 循环

C++ 的 `for` 语句非常不同。

```
1 var i: integer;
2
3 begin
4     for i:= 1 to 10 do
5         begin
6             write(i, ' ');
7         end;
8 end.

1 for (int i = 1; i <= 10; i++) {
2     std::cout << i << " ";
3 }
```

注释：

1. `for (int i = 1; i <= 10; i++){` 这一行语句很多，`for` 中有三个语句。
2. 第一个语句 `int i = 1;` 此时声明一局部变量 `i` 并初始化。（这个设计比 Pascal 要合理得多。）
3. 第二个语句 `i <= 10;` 作为判断循环是否继续的标准。
4. 第三个语句 `i++`，在每次循环结尾执行，意思大约就是 Pascal 中的 `inc(i)`，此处写成 `++i` 也是一样的。`i++` 与 `++i` 的区别请参考其他资料。

repeat until 与 do while 循环

注意，`repeat until` 与 `do while` 是不同的，请对比以下代码

```
1 var i: integer;
2
3 begin
4     i := 1;
5     repeat
6         write(i, ' ');
7         inc(i);
8     until i = 11;
9 end.

1 int i = 1;
2 do {
3     std::cout << i << " ";
4     i++;
5 } while (i <= 10);
```

循环控制 Loop Control

C++ 中 `break` 的作用与 Pascal 是一样的，退出循环。

而 `continue` 也是一样的，跳过当前循环，进入下一次循环（回到开头）。

数组与字符串 Array and String

不定长数组：标准库类型 Vector

C++ 标准库中提供了 `vector`，相当于不定长数组，调用前需导入库文件。

```

1 #include <LTiostream>
2 #include <LTvector> // 导入 vector 库
3
4 int main() {
5     std::vector<int> a; // 声明 vector a 并定义 a 为空 vector 对象
6     int n;
7
8     std::cin >> n;
9     // 读取 a
10    for (int i = 0; i < n; i++) {
11        int t;
12        std::cin >> t;
13        a.push_back(t); // 将读入的数字 t, 放到 vector a 的末尾; 该操作复杂度 O(1)
14        /* 这里不能使用下标访问来赋值, 因为声明时, a 大小依然为空,
15        此处使用 `a[i] = t;` 是错误做法。
16        */
17    }
18
19    // 将读入到 a 中的所有数打印出
20    for (int i = 0; i < n; i++) {
21        std::cout << a[i] << ", "; // !注意, a 中第一个数是 a[0];
22        // 如果下标越界, 它会返回一个未知的值 (溢出), 而不会报错
23    }
24    std::cout << std::endl;
25
26    return 0;
27 }

```

C++ 访问数组成员, 与 Pascal 类似, 不过有很重要的区别: 数组的第一项是 `a[0]`, 而 Pascal 中是可以自行指定的。

扩展阅读:

- [序列式容器 - OI Wiki](#)

字符串: 标准库类型 String

C++ 标准库中提供了 `string`, 与 `vector` 可以进行的操作有些相同, 同样需要导入库文件。

```

1 #include <LTiostream>
2 #include <LTstring>
3
4 int main() {
5     std::string s; // 声明 string s
6
7     std::cin >> s; // 读入 s;
8     // 读入时会忽略开头所有空格符 (空格、换行符、制表符), 读入的字符串直到下一个空格符为止。
9
10    std::cout << s << std::endl;
11
12    return 0;
13 }

```

扩展阅读:

- [string - OI Wiki](#)

C 风格数组 Array

如果要用不定长的数组请用 `Vector`, 不要用 C 风格的数组。

C 风格的数组与指针有密切关系, 所以此处不多展开。

扩展阅读：

- [数组 - OI Wiki](#)

重要不同之处 Differences

变量作用域 Scope：全局变量与局部变量

C++ 几乎可以在任何地方声明变量。

以下对于 C++ 的变量作用域的介绍摘自 [变量作用域 - OI Wiki](#)：

作用域是变量可以发挥作用的代码块。

变量分为全局变量和局部变量。在所有函数外部声明的变量，称为全局变量。在函数或一个代码块内部声明的变量，称为局部变量。

全局变量的作用域是整个文件，全局变量一旦声明，在整个程序中都是可用的。

局部变量的作用域是声明语句所在的代码块，局部变量只能被函数内部或者代码块内部的语句使用。

由一对大括号括起来的若干语句构成一个代码块。

```
1 int g = 20; // 声明全局变量
2 int main() {
3     int g = 10; // 声明局部变量
4     printf("%d\n", g); // 输出 g
5     return 0;
6 }
```

在一个代码块中，局部变量会覆盖掉同名的全局变量，比如上面的代码输出的 `g` 就是 `10` 而不是 `20`。为了防止出现意料之外的错误，请尽量避免局部变量与全局变量重名的情况。

在写 Pascal 过程/函数时，容易忘记声明局部变量 `i` 或者 `j`，而一般主程序里会有循环，于是大部分情况下 `i` 与 `j` 都是全局变量，于是，在这种情况下，过程/函数中对 `i` 操作极易出错。更要命的是，如果忘记声明这种局部变量，编译器编译不报错，程序可以运行。（有很多难找的 bug 就是这么来的。）

所以，在使用 C++ 时，声明变量，比如循环中使用的 `i`，**不要用全局变量，能用局部变量就用局部变量**。如果这么做，不用担心函数中变量名（比如 `i`）冲突。

► 额外注

C++ 可以自动转换类型

```
1 int i = 2;
2 if (i) { // i = 0 会返回 false, 其余返回 true
3     std::cout << "true";
4 } else {
5     std::cout << "false";
6 }
```

不光是 `int` 转成 `bool`，还有 `int` 与 `float` 相互转换。在 Pascal 中可以把整型赋给浮点型，但不能反过来。C++ 没有这个问题。


```

1 int a;
2 a = 3.2;      // 此时 a = 3
3 float b = a;  // 此时 b = 3.0

```

区分 / 是整除还是浮点除法，是通过除数与被除数的类型判断的

```

1 float a = 32 / 10;    // 32/10 的结果是 3（整除）；a = 3.0
2 float b = 32.0 / 10;  // 32.0/10 的结果是 3.2；b = 3.2

```

`pow(a, b)` 计算，该函数返回的是浮点型，如果直接用来计算整数的幂，由于有自动转换，不需要担心它会报错

```

1 int a = pow(2, 3);    // 计算 2^3

```

还有 `char` 与 `int` 之间相互转换。

```

1 char a = 48;          // ASCII 48 是 '0'
2 int b = a + 1;         // b = 49
3 std::cout << (a == '0'); // true 输出 1

```

其实 C++ 中的 `char` 与 `bool` 本质上是整型。

扩展阅读：

- [隐式转换 - cppreference.com](http://cppreference.com) 注意内容可能过于专业

C++ 很多语句有返回值：以如何实现读取数量不定数据为例

有些时候需要读取到数据结束，比如，求一组不定数量的数之和（数据可以多行），直到文件末尾，实现方式是

► 文件末尾 EOF

```

1 #include <fstream>
2
3 int main() {
4     int sum = 0, a = 0;
5
6     while (std::cin >> a) {
7         sum += a;
8     }
9     std::cout << sum << std::endl;
10
11     return 0;
12 }

```

实现原理：`while (std::cin >> a)` 中 `std::cin >> a` 若在输入有问题或遇到文件结尾时，会返回 `false`，使得循环中断。

函数 Function：C++ 只有函数没有过程但有 `void`，没有函数值变量但有 `return`。

Pascal 函数与 C++ 函数对比示例：

```

1 function abs(x:integer):integer;
2 begin
3     if x < 0 then
4         begin
5             abs := -x;
6         end
7     else
8         begin
9             abs := x;
10        end;
11 end;

```

```

1 int abs(int x) {
2     if (x < 0) {
3         return -x;
4     } else {
5         return x;
6     }
7 }

```

C++ 中函数声明 `int abs`，就定义了 `abs()` 函数且返回值为 `int` 型（整型），函数的返回值就是 `return` 语句给出的值。

如果不想有返回值（即 Pascal 的「过程」），就用 `void`。`void` 即「空」，什么都不返回。

```

1 var ans: integer;
2
3 procedure printAns(ans:integer);
4 begin
5     writeln(ans);
6 end;
7
8 begin
9     ans := 10;
10    printAns(ans);
11 end.

```

```

1 #include <LTIostream>
2
3 void printAns(int ans) {
4     std::cout << ans << std::endl;
5
6     return;
7 }
8
9 int main() {
10     int ans = 10;
11     printAns(ans);
12
13     return 0;
14 }

```

C++ 的 `return` 与 Pascal 中给函数变量赋值有一点非常大的不同。C++ 的 `return` 即返回一个值，执行完这个语句，函数就执行结束了；而 Pascal 中给函数变量赋值并不会跳出函数本身，而是继续执行。于是，如果 Pascal 需要某处中断函数/过程，就需要一个额外的命令，即 `exit`。而 C++ 则不需要，如果需要在某处中断，可以直接使用 `return`。比如（由于实在想不出来简短且实用的代码，所以就先这样）

```

1 #include <Iostream>
2
3 void printWarning(int x) {
4     if (x >= 0) {
5         return; // 该语句在此处相当于 Pascal 中的 `exit;`
6     }
7     std::cout << "Warning: input a negative number.";
8 }
9
10 int main() {
11     int a;
12
13     std::cin >> a;
14     printWarning(a);
15
16     return 0;
17 }

```

而在某种意义上，前面的 `abs` 函数，这样才是严格等效的

```

1 function abs(x:integer):integer;
2 begin
3     if x < 0 then
4         begin
5             abs := -x; exit; // !注意此处
6         end
7     else
8         begin
9             abs := x;  exit; // !注意此处
10        end;
11 end;

```

```

1 int abs(int x) {
2     if (x < 0) {
3         return -x;
4     } else {
5         return x;
6     }
7 }

```

▼ 特别提醒

C++ 中 `exit` 是退出程序；不要顺手把 `exit` 打上去，要用 `return`！

C++ 把函数和过程统统视作函数，连 `main` 都不放过，比如写 `int main`，C++ 视 `main` 为一个整型的函数，这里返回值是 0。它是一种习惯约定，返回 0 代表程序正常退出。

也许你已经猜到了，`main(int argc, char const *argv[])` 中的参数就是 `int argc` 与 `char const *argv[]`，不过意义请参考其他资料。

在函数中传递参数 Passing Parameters to Functions

C++ 中没有 Pascal 的 `var` 关键字可以改变传递的参数，但是 C++ 可以使用引用和指针达到同样的效果。

```

1 var a, b: integer;
2
3 procedure swap(var x,y:integer);
4 var temp:integer;
5 begin
6     temp := x;
7     x := y;
8     y := temp;
9 end;
10
11 begin
12     a := 10; b:= 20;
13     swap(a, b);
14     writeln(a, ' ', b);
15 end.

```

```

1 // 使用指针的代码
2 #include <LTIostream>
3
4 void swap(int* x, int* y) {
5     int temp;
6     temp = *x;
7     *x = *y;
8     *y = temp;
9 }
10
11 int main() {
12     int a = 10, b = 20;
13     swap(&a, &b);
14     std::cout << a << " " << b;
15
16     return 0;
17 }

```

注意，此处 C++ 代码 **涉及指针问题**。指针问题还是很麻烦的，建议去阅读相关资料。

```

1 // 使用引用的代码
2 #include <LTIostream>
3
4 void swap(int& x, int& y) {
5     int temp;
6     temp = x;
7     x = y;
8     y = temp;
9 }
10
11 int main(int argc, char const* argv[]) {
12     int a = 10, b = 20;
13     swap(a, b);
14     std::cout << a << " " << b;
15
16     return 0;
17 }

```

注意，此处 C++ 代码 **涉及引用相关类型问题**。在用引用调用一些 STL 库、模板库的时候可能会遇到一些问题，这时候需要手动声明别类型。具体资料可以在《C++ Primer》第五版或者网络资料中自行查阅。

C++ 中函数传递参数还有其它方法，其中一种是 **直接使用全局变量传递参数**，如果不会用指针，可以先用这种方法。但是这种方法的缺陷是没有栈保存数据，**没有办法在递归函数中传参**。（除非手写栈，注意，手写栈也是一种突破系统栈限制的方法。）

C++ 标准库与参考资料 Reference

千万不要重复造轮子（除非为了练习），想要自己动手写一个功能出来之前，先去看看有没有这个函数或者数据结构。

C++ 标准库

C++ 标准库中 `<algorithm>` 有很多有用的函数比如快排、二分查找等，可以直接调用。请参考这个页面：[STL 算法 - OI Wiki](#)。

还有 STL 容器，比如数组、向量（可变大小的数组）、队列、栈等，附带很多函数。请参考这个页面：[STL 容器简介 - OI Wiki](#)。

如果要找关于字符串操作的函数见

- [std::basic_string - cppreference.com](#)
- [<string> - C++ Reference](#)

C/C++ 的指针是很灵活的东西，如果想要彻底理解指针，建议找本书或者参考手册仔细阅读。

- [指针 - OI Wiki](#)

错误排查与技巧

- [常见错误 - OI Wiki](#)
- [常见技巧 - OI Wiki](#)

C++ 语言资料

- [学习资源 - OI Wiki](#)
- [cppreference.com](#) - 最重要的 C/C++ 参考资料
- [C++ 教程 - 菜鸟教程](#)
- [C++ Language - C++ Tutorials](#)
- [Reference - C++ Reference](#)
- [C++ Standard Library - Wikipedia](#)
- [The Ultimate Question of Programming, Refactoring, and Everything](#)
- [Google C++ Style Guide](#)

后记

写到这里，很多同学会觉得这一点都不急救啊，有很多东西没有提到啊。那也是没办法的事情。

虽然是为了急救，但很多东西像怎么把字符串转化为数字，怎么搜索字符串中的字符，这些东西也不适合一篇精悍短小的急救帖，如果把这些都写出来，那就是 C++ 入门教程，所以请充分利用本 Wiki、参考手册与搜索引擎。

需要指出的一点是，上面说 C++ 的语法，其实有很多语法是从 C 语言来的，标题这么写比较好——《Pascal 转 C/C++ 急救帖》。

Pascal 在上个世纪后半叶是门很流行的语言，它早于 C 语言，不过随着 UNIX 系统的普及，微软使用 C 语言，现在 Pascal 已经成为历史了。Pascal 后期发展也是有的，比如 Free Pascal 这个开源编译器项目，增加面向对象的特性（Delphi 语言）。Pascal 目前的用处除了在信息竞赛外，有一个特点是其他语言没有的——编译支持非常非常多老旧机器，比如 Gameboy 这种上个世纪的任天堂游戏机，还有一个用处就是以伪代码的形式（Pascal 风格的伪代码）出现在各种教科书中。

最后，Pascal 的圈子其实很小，C/C++ 的圈子很大，帮助手册与教程很多很全，一定要掌握好英语。世界上还有很多很多编程语言，而计算机这门学科与技术不光是信息竞赛和编程语言。

本文 Pascal 语言的参考文献

- [Lazarus wiki](#)
- [Free Pascal Reference guide](#)

附 A: Pascal 与 C++ 运算符与数学函数语法对比表 Pascal vs C++ Operator Syntax Table

仅包括最常用的运算符与函数。

基本算术

	Pascal	C++
加法	<code>a + b</code>	<code>a + b</code>
减法	<code>a - b</code>	<code>a - b</code>
乘法	<code>a * b</code>	<code>a * b</code>
整除	<code>a div b</code>	<code>a / b</code>
浮点除法	<code>a / b</code>	<code>a / b</code>
取模	<code>a mod b</code>	<code>a % b</code>

逻辑

	Pascal	C++
非	<code>not(a)</code>	<code>!a</code>
且	<code>a and b</code>	<code>a && b</code>
或	<code>a or b</code>	<code>a b</code>

比较

	Pascal	C++
相等	<code>a = b</code>	<code>a == b</code>
不等	<code>a <> b</code>	<code>a != b</code>
大于	<code>a > b</code>	<code>a > b</code>
小于	<code>a < b</code>	<code>a < b</code>
大于等于	<code>a >= b</code>	<code>a >= b</code>
小于等于	<code>a <= b</code>	<code>a <= b</code>

赋值

Pascal	C++
<code>a := b</code>	<code>a = b</code>
<code>a := a + b</code>	<code>a += b</code>
<code>a := a - b</code>	<code>a -= b</code>
<code>a := a * b</code>	<code>a *= b</code>
<code>a := a div b</code> 或 <code>a := a / b</code>	<code>a /= b</code>
<code>a := a mod b</code>	<code>a %= b</code>

自增/自减

Pascal C++

自增 `inc(a) a++`

自增 `inc(a) ++a`

自减 `dec(a) a--`

自减 `dec(a) --a`

数学函数

使用需要导入 `<cmath>` 库。

	Pascal	C++
绝对值	<code>abs(a)</code>	<code>abs(a)</code> (整数)
绝对值	<code>abs(a)</code>	<code>fabs(a)</code> (浮点数)
	N/A (*)	<code>pow(a, b)</code>
截断取整	<code>trunc(a)</code>	<code>trunc(a)</code>
近似取整	<code>round(a)</code>	<code>round(a)</code>

*Extended Pascal 中有 `a**b` 不过需要导入 `Math` 库。

其他函数请参考：

- [常用数学函数 - cppreference.com](http://cppreference.com)

附 B：文章检索 Index

按 C++ 语句语法索引。

- [基本语法](#)
- [变量](#)
 - [数据类型](#)
 - [常量声明](#)
 - [作用域](#)
- [运算符](#)
- [if 语句](#)
 - `if`
 - `else`
- [循环语句](#)
 - [for 语句](#)
 - [while 语句](#)
 - [do while 语句](#)
 - [break, continue](#)
- [函数](#)
 - [函数定义, return](#)
 - [函数传参](#)
- [数组与字符串](#)
 - [不定长数组 Vector](#)
 - [C 风格数组](#)
 - [字符串 String](#)
- [资料](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[lr1d](#), [kexplorning](#), [lvneg1](#), [StudyingFather](#), [billchenchina](#), [CCXXXI](#), [Chrogeek](#), [Draudnaut](#), [Enter-tainer](#), [FinParker](#), [hly1204](#), [Konano](#), [megakite](#), [ouuan](#), [qz-cqy](#), [ShizuhaAki](#), [shuzhouliu](#), [sshwy](#), [Suyun514](#), [Tiphereth-A](#), [TrickEye](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用